Análisis de módulos de visualización

Este documento describe cómo se inicia y presenta la aplicación, cómo se procesan las interacciones y cómo se editan escenarios desde la interfaz. En los módulos que dibujan rutas se analiza la representación en pantalla; el cálculo de caminos y el recorrido de pickers quedan fuera de este análisis.

Las cotas se expresan en función de la cantidad de elementos que la interfaz dibuja o recorre. Se separa el trabajo que ocurre una sola vez o cuando cambian los datos del que puede repetirse en cada cuadro de pantalla.

Variables

- A: cantidad de píxeles del lienzo de referencia; F: cuadros dibujados por segundo.
- R: racks; Z: zonas de despacho; D: puertas; N: productos; P: pickers; W: waves.
- C: celdas de la grilla; E: elementos guardados por el editor; H: entradas de historial/recogidas recorridas al dibujar.
- T: tareas pendientes y objetivos visibles; L: longitud de texto procesado.

1. Inicio de la aplicación: Main

Idea y estructuras

Main prepara las rutas de importación y decide con qué datos inicia la aplicación. Puede recibir un escenario por argumento, restaurar el autoguardado o generar un escenario inicial.

Paso a paso

1. Ajusta la configuración de DPI y agrega las carpetas del proyecto al sys.path.
2. Lee el argumento --escenario; si no se indicó, busca el autoguardado.
3. Si no pudo obtener datos válidos, llama a generar_escenario.
4. Importa App, crea la interfaz y entra al bucle principal.

Complejidad

La coordinación propia de main es O(1) en cantidad de elementos. La carga, validación o generación inicial se delega a Persistence y Scenario; la creación de la simulación e interfaz también tiene sus propios costos. El espacio de Main es O(1) adicional a los objetos entregados a la aplicación.

2. Control de interfaz: App

Idea y estructuras

App conecta la ventana Pygame con la sesión, la animación, la vista del mapa, los paneles y la pantalla generadora. Mantiene un lienzo lógico de tamaño fijo que escala a la ventana real, un reloj y los objetos de la interfaz.

Paso a paso

1. Inicializa Pygame, calcula el tamaño de ventana y crea el lienzo lógico.
2. Construye Sesion, fuentes, pincel, animación, vista y paneles.

3. Convierte las coordenadas del mouse de ventana a coordenadas lógicas y distribuye eventos de teclado, mouse y rueda.
4. Ejecuta acciones: iniciar, pausar, avanzar, cargar/guardar, abrir el generador o cambiar opciones visuales.
5. En cada cuadro actualiza la animación o avanza la sesión cuando corresponde; dibuja la vista activa, escala el lienzo y presenta el cuadro.

Complejidad

El despacho de un evento simple cuesta O(1); una acción de carga, guardado o validación hereda el costo del módulo que la realiza. El costo por cuadro es la suma de dibujar mapa, paneles, alertas y escalar el lienzo: O(A + trabajo_vistas). Se repite hasta F veces por segundo. El lienzo y las superficies base usan O(A) espacio; los demás objetos ocupan espacio proporcional al estado que muestran.

3. Interpolación visual: Animacion

Idea y estructuras

Animacion guarda la velocidad seleccionada, el progreso entre dos ticks, las posiciones anteriores de los pickers y el estado de pausa. No modifica la lógica de la simulación: interpola posiciones para dibujarlas suavemente.

Paso a paso

1. Al comenzar una transición guarda las posiciones anteriores y pone el progreso en cero.
2. avanzar(dt) incrementa el progreso según el tiempo y la velocidad, limitado a uno.
3. posicion_visual interpola entre posición previa y actual; al finalizar devuelve la posición actual.
4. Acelerar, frenar, pausar y continuar actualizan campos escalares.

Complejidad

Las actualizaciones y consultas individuales cuestan O(1). Para P pickers, preparar posiciones para una transición cuesta O(P) y ocupa O(P) espacio.

4. Paneles: PanelLateral y PanelWaves

Idea y estructuras

PanelLateral dibuja los controles, selectores de estrategia, velocidad y tarjetas de pickers. PanelWaves muestra métricas, progreso, estado de waves y el registro. Ambos leen una instantánea de Sesion y usan controles y colores de Widgets y Config.

Paso a paso

1. PanelLateral construye una lista fija de botones y dibuja sus estados activos.
2. Muestra hasta cuatro tarjetas de picker; cada tarjeta calcula la carga recogida del trabajo actual.
3. PanelWaves calcula tick, entregas, progreso de unidades, asignaciones y búsquedas reportadas.
4. Ordena waves por prioridad de visualización, dibuja hasta cinco y permite desplazarse por las restantes.
5. Dibuja las últimas cuatro líneas del registro.

Complejidad

PanelLateral dibuja una cantidad fija de controles y hasta cuatro tarjetas. El cálculo de carga recorre las recogidas del picker visible: O(H) por tarjeta, hasta O(4H) por cuadro.

PanelWaves ordena las waves en O(W log W). Para las métricas puede volver a sumar recogidas de pickers por cada wave, hasta O(W·P·H); además calcula el progreso de hasta cinco waves visibles. Su espacio auxiliar es O(W) por el ordenamiento. Estos costos se pagan al dibujar el panel, no al avanzar el algoritmo de simulación.

5. Vista del mapa: VistaMapa

Idea y estructuras

VistaMapa combina una capa estática del warehouse con overlays dinámicos: capacidad de despacho, productos, pickers, objetivos, historial visual e intersecciones. Mantiene una Grilla, una fábrica de sprites y una superficie de fondo reutilizable.

Paso a paso

1. Al cambiar de escenario, recalcular actualiza la grilla y reconstruye la capa estática.
2. En cada cuadro dibuja encabezado y capacidad por zona.
3. Dibuja intersecciones si están activadas, productos según su estado y marcadores visuales de tareas.
4. Dibuja trazas/recorridos ya informados por la simulación y los pickers en sus posiciones interpoladas.
5. Completa leyenda, estados y tooltip.

Complejidad

La reconstrucción de grilla/capa ocurre al cambiar el warehouse y cuesta lo que cuesta recorrer sus celdas y elementos en CapaMapa. El overlay por cuadro recorre productos y pickers, además de las posiciones de estela y tareas dibujadas: O(N + P + H + T + I), donde I es la cantidad de intersecciones si se muestran. La superposición de tarjetas de zonas compara ubicaciones ya dibujadas y puede llegar a $O(Z^2)$. El espacio adicional es O(A + P + N) para superficies, sprites y estado visual.

6. Fondo estático: CapaMapa

Idea y estructuras

CapaMapa arma una superficie con suelo, pasillos y sentidos, zonas/puertas, racks, borde y efectos visuales. Los colores se definen como constantes; la superficie se reutiliza hasta que se reconstruye el warehouse.

Paso a paso

1. construir crea o limpia la superficie del mapa.
2. Dibuja suelo y pasillos, incluyendo marcas de sentido.
3. Dibuja zonas de despacho y sus puertas, después racks y detalles.
4. dibujar copia la superficie ya construida al lienzo principal.

Complejidad

Sea C_m el número de celdas cubiertas por pasillos y elementos. Construir la capa cuesta O(A + C_m + R + Z + D) por inicializar/dibujar la superficie y recorrer la geometría. El blit posterior cuesta proporcional al área de superficie, O(A), según el backend gráfico. La superficie cacheada ocupa O(A) píxeles.

7. Pantalla generadora: PantallaGenerador

Idea y estructuras

La pantalla ofrece pestañas Automático y Manual. Mantiene los parámetros del generador, modo de ubicación, controles, scroll, el editor manual y una vista previa. Devuelve acciones a App; no ejecuta la simulación.

Paso a paso

1. Inicializa valores y controles con límites para cada parámetro.
2. En Automático procesa steppers y modos, sincroniza la vista previa cuando cambia la clave de parámetros y calcula si la configuración puede generarse.
3. En Manual reenvía los eventos al editor, sincroniza cantidades de elementos y actualiza validación tras un breve retraso.
4. Dibuja el formulario, la miniatura o el mapa, la lista de pickers, acciones y errores.
5. Al confirmar, devuelve parámetros automáticos o datos del mapa manual a App.

Complejidad

La mayoría de controles y parámetros son una cantidad fija, por lo que su procesamiento cuesta O(1) respecto del tamaño del warehouse. La vista automática se regenera solo si cambian sus parámetros; su costo es el de generar_vista_previa más dibujar R + Z + D + N + P elementos. En Manual, sincronizar la lista cuesta O(P) y validar delega en ModeloEscenario/Scenario. Dibujar la lista de pickers cuesta O(P) aunque solo se pinten filas visibles. El estado de la pantalla usa O(E + N + P) además de la vista previa.

8. Editor de escenario: EditorEscenario y cuadros modales

Idea y estructuras

EditorEscenario traduce clicks, arrastres y teclas a operaciones de ModeloEscenario. CuadroProducto y CuadroPesoZona recogen datos numéricos; _ElementoVista adapta diccionarios a objetos geométricos para CapaMapa. La rejilla y la capa del mapa se cachean por separado.

Paso a paso

1. Convierte la posición del mouse a celda y registra herramienta, punto inicial y punto actual del arrastre.
2. Al soltar, calcula el rectángulo y delega validación/alta en el modelo.
3. Para productos o peso de despacho abre el cuadro modal y valida los números ingresados.
4. Permite mover pickers, borrar elementos, deshacer y seleccionar sentidos de pasillos.
5. Marca cambios; después de una pausa solicita validación y refresca la capa si cambió la versión del modelo.
6. Dibuja mapa cacheado, grilla, productos, pickers, selección del arrastre, cursor y mensajes.

Complejidad

La traducción de evento es O(1) salvo operaciones del modelo, que recorren elementos o copian el estado para deshacer: hasta O(E + N) por acción. La validación se ejecuta luego del retraso y su costo incluye comprobar geometría y validar el escenario, con construcción de grafo. Reconstruir la capa cuesta O(A + C_m + E) y solo ocurre cuando cambia el modelo. El dibujo dinámico recorre N + P elementos y la lista de pickers por celda, O(P). El historial conserva hasta 100 instantáneas, así que ocupa O(100·E) más los productos.

9. Vista previa automática: VistaPreviaEscenario

Idea y estructuras

Muestra cómo quedaría el layout automático sin crear pedidos ni construir el grafo. Guarda la última clave de parámetros, los datos resultantes, cualquier error y la transformación entre celdas y píxeles.

Paso a paso

1. actualizar compara los parámetros con la clave previa; si no cambiaron, reutiliza los datos.
2. Si cambiaron, llama a generar_vista_previa y guarda el resultado o el error.
3. dibujar calcula el tamaño de celda y convierte coordenadas de racks, zonas, puertas, productos y pickers.
4. Dibuja cada tipo de elemento con primitivas simples.

Complejidad

Actualizar sin cambios cuesta O(1). Al cambiar parámetros se paga la generación de vista previa. Dibujarla cuesta O(R + Z·D + D + N + P): la búsqueda de puerta correspondiente puede recorrer puertas por cada zona. Guarda O(R + Z + D + N + P) datos del último resultado.

10. Conversión de coordenadas: Grilla

Idea y estructuras

Grilla guarda tamaño de celda, origen del mapa y dimensiones lógicas. Convierte una celda a rectángulo o centro en píxeles, y una posición del mouse a celda.

Complejidad

La inicialización y cada conversión (centro, rect, celda_en) cuestan O(1) y usan O(1) espacio. celda_en también comprueba que la coordenada esté dentro de los límites.

11. Controles reutilizables: Widgets

Idea y estructuras

El módulo reúne la posición/estado global del mouse, Boton y Button, Stepper, ToolButton, StatusBar, Alerta, ListaPickersScroll, PanelValidacion, Badge y funciones de dibujo. Los controles conservan rectángulos, textos, estado activo y límites.

Paso a paso

1. Las funciones globales actualizan el mouse lógico y el estado del botón presionado.
2. Botones y steppers comprueban clicks y ajustan su valor dentro de límites.
3. Alerta envuelve el texto, anima su aparición y dibuja caja, progreso y cierre.

4. ListaPickersScroll calcula el desplazamiento, detecta eliminación y dibuja filas visibles.
5. PanelValidacion muestra el estado y hasta cuatro errores; las funciones comunes dibujan badges y botones.

Complejidad

Botones, steppers, etiquetas y controles de estado cuestan O(1) por evento/dibujo. Envolver un texto cuesta según su longitud L. ListaPickersScroll.clic_eliminar y dibujar recorren la lista para identificar filas visibles, O(P); el dibujo efectivo omite las que quedan fuera del clip. El panel de validación dibuja como máximo cuatro errores, O(1). El espacio de cada control es O(1) y la lista ocupa O(P).

12. Recursos gráficos: Sprites y SpritesProducto

Idea y estructuras

FabricaSprites genera montacargas/robots, sombras y halos; FabricaProductos genera cajas por tamaño y estado, además de auras. Ambas guardan las superficies generadas en diccionarios cacheados por sus parámetros visuales.

Paso a paso

1. La vista pide un sprite usando color, dirección, radio, carga, estado o intensidad.
2. Si la combinación ya existe, devuelve la superficie cacheada.
3. Si no, dibuja formas y degradados en una superficie temporal, escala/rota cuando corresponde y la guarda.
4. Las llamadas posteriores reutilizan esa superficie.

Complejidad

La búsqueda en caché cuesta O(1) esperado. Crear un sprite o aura de lado s cuesta $O(s^2)$ por el trabajo sobre superficies de píxeles (con varias pasadas constantes). La memoria acumulada es proporcional a la suma del área de las variantes cacheadas; depende de combinaciones usadas y no tiene un límite explícito.

13. Primitivas de dibujo y fuentes: Pincel

Idea y estructuras

Fuentes crea las tipografías compartidas. Pincel concentra operaciones gráficas sobre la superficie principal: texto, círculos, anillos, rectángulos translúcidos y recorte de texto.

Complejidad

Crear fuentes ocurre al iniciar la interfaz y depende del backend. Dibujar texto o una forma cuesta proporcional a los píxeles que rasteriza. recortar elimina caracteres hasta que el texto entra en el ancho solicitado; en el peor caso hace varias mediciones de prefijo, hasta $O(L^2)$ si cada medición vuelve a recorrer el texto. Las superficies temporales de transparencia usan O(area) píxeles.

14. Diálogos nativos: Dialogos

Idea y estructuras

abrir y guardar crean un diálogo Tkinter temporal para elegir un archivo JSON. El resto de la interfaz no depende de Tkinter.

Complejidad

El trabajo del código es O(1) más la creación/cierre de la ventana nativa. El tiempo de interacción del usuario y del sistema operativo no tiene una cota algorítmica útil. No conserva estado entre llamadas.

15. Constantes visuales: Config

Idea y estructuras

Config contiene dimensiones de ventana/paneles, áreas de mapa, colores, fuentes, velocidad, temporizadores, estados y límites de visualización. Son tuplas y diccionarios constantes importados por las vistas.

Complejidad

Consultar una constante o buscar un color/estado cuesta O(1) esperado. La carga inicial ocupa O(K) espacio, donde K es la cantidad fija de constantes; no realiza cálculos de escenario ni dibujo.

Resumen

Módulo	Costo dominante	Espacio auxiliar
Main	O(1) propio más carga/generación inicial	O(1) propio
App	O(A + trabajo_vistas) por cuadro	O(A + estado visual)
Animacion	O(1) por picker; O(P) por transición	O(P)
PanelLateral	O(4H) por cuadro	O(1) adicional
PanelWaves	O(W·P·H + W log W) por cuadro	O(W)
VistaMapa	O(N + P + H + T + I + Z^2) por cuadro	O(A + P + N)
CapaMapa	construir O(A + C_m + R + Z + D)	O(A) cacheado
PantallaGenerador	vista/validación delegada; listas O(P)	O(E + N + P)
EditorEscenario	acción O(E + N); capa solo al cambiar	historial O(100·E + N)
VistaPreviaEscenario	O(R + Z·D + D + N + P) por dibujo	O(R + Z + D + N + P)
Grilla	O(1) por conversión	O(1)
Widgets	controles O(1); lista O(P)	O(P)
Fábricas de sprites	hit O(1); creación O(s^2)	suma de superficies cacheadas
Pincel	proporcional a píxeles/texto; recorte hasta O(L^2)	superficies temporales
Dialogos	O(1) del programa más espera del sistema	O(1)
Config	consultas O(1)	O(K) fijo

Los costos por cuadro se repiten a la frecuencia de refresco de la interfaz. Las capas y previews cacheadas evitan volver a generar elementos estáticos mientras no cambien el mapa o sus parámetros. La interfaz solo presenta el estado; las decisiones de asignación, navegación y avance pertenecen a los módulos de simulación.